

Relational Databases & SQL

Digital Strategies for the
Social Sciences

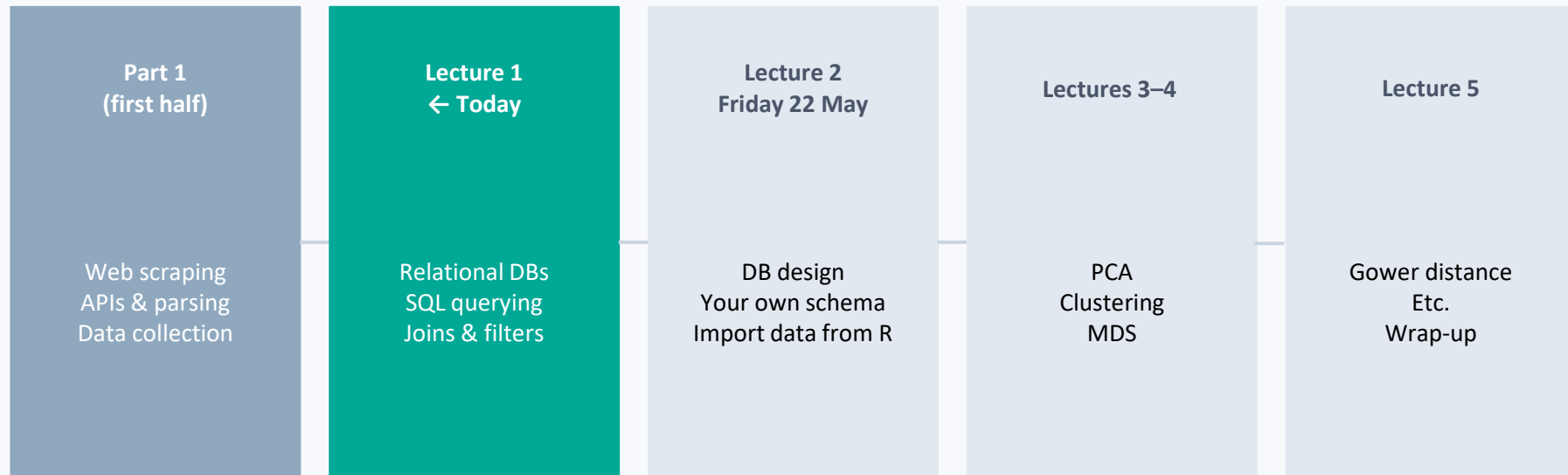
Carl Nordlund

www.carlnordlund.net



Where We Are in This Course

SQL: good to know, and useful bridge between data collection and data analysis



The data you query in SQL today will reappear in Lectures 3–5

Today's Agenda

01

Why Databases?

The CSV problem
Relational model
ER diagrams & keys

02

SQLite in R

DBI · RSQLite
Connecting & exploring
Two query styles

03

SQL Syntax

SELECT · WHERE
GROUP BY · JOINS
CASE WHEN

04

Lab 1

Explore the music DB
Write your own queries
EDA preview

How Do You Currently Store Your Scraped Data?

Think about your last scraping exercise!

Example:

You scraped a music platform!

Artists have names, countries, and genres.

Each artist has dozens of tracks with durations, audio features, and popularity scores.

Where does all of this data live?

artists.csv

tracks.csv

genres.csv

merged_final_v3.csv

?!?

What problems appear when the project grows, or when a collaborator needs to update the data?

The Problem with Flat Files

What goes wrong when data grows

tracks.csv (flat file: one row per track)

track_id	title	artist_name	artist_genre
1	Bohemian Rhapsody	Queen	Rock
2	We Will Rock You	Queen	Rock
3	Radio Ga Ga	Queen	Rock
4	Like a Prayer	Madonna	Pop
5	Vogue	Madonna	Pop

What goes wrong:

- Duplication — every artist field copied for every track
- Update anomalies — change a country, update thousands of rows
- Merge errors — manual joins create silent mismatches
- No integrity — a typo silently breaks file relationships
- One-to-many relationships are impossible to represent cleanly

Queen's genre stored three times — and there may be thousands of Queen tracks in the full dataset!

Replacing 'Rock' with 'Symphony rock'? Then you have to do it in every line!

The Relational Model

One entity, one table: link tables/entities with keys



One table per entity

Artists go in artists
Tracks go in tracks
One row is one instance. Entity types
are never mixed in the same table.



Keys replace copies

Instead of copying 'Queen' into every
track row, the tracks table holds only
the artist's ID. One number links to all
the details.



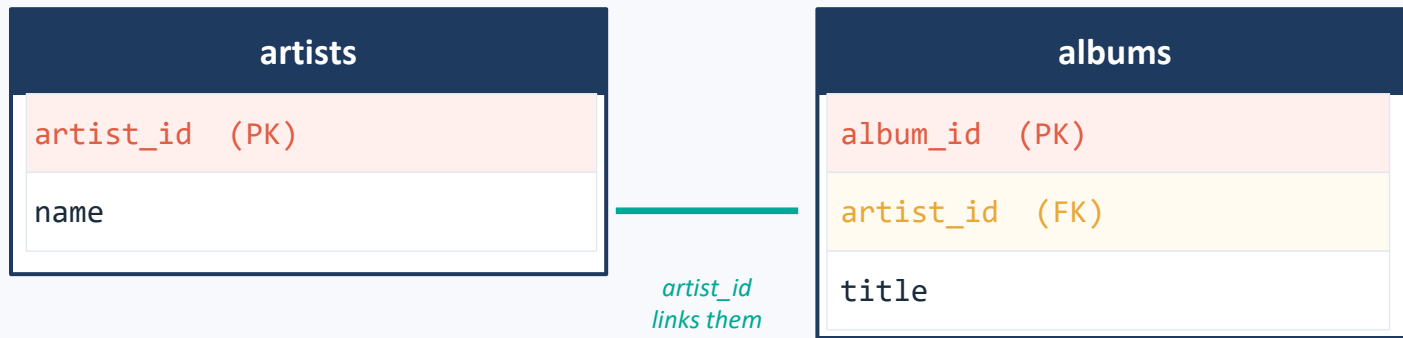
Normalisation

Every fact is stored exactly once.
Update a name in one row and
everything that references that ID
reflects the change automatically.

This model has underpinned data storage since the 1970s and remains the standard in research, government, and industry!

Primary Keys & Foreign Keys

The mechanism that links tables without duplicating data



If `artist_id = 47` in `albums`, there must be a row with `artist_id = 47` in `artists`.

This guarantee is called **referential integrity**.

The database enforces it — not your code.

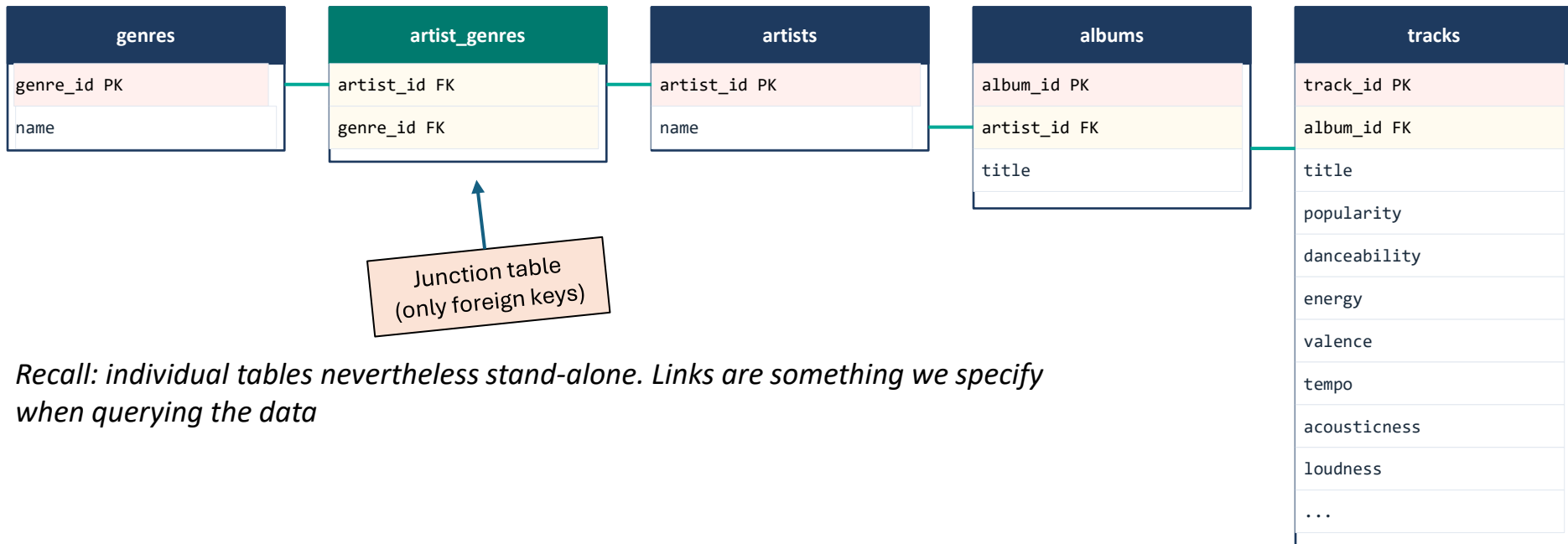
That said: tables are nevertheless stand-alone! The link is something we (could) include when querying

PK Primary Key — uniquely identifies each row. No two rows share a PK value.

FK Foreign Key — references a PK in another table. Could be many rows with same value for a FK. The artist 'Queen' has many albums: each album a unique `album_id` but the same `artist_id` (that points to the same row in 'artists').

The Music Database — Entity-Relationship Diagram

Five tables · Rich audio features for EDA · Distributed as a single .sqlite file



Recall: individual tables nevertheless stand-alone. Links are something we specify when querying the data

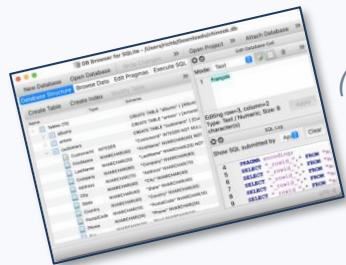
Audio features (danceability, energy, valence, tempo, acousticness...) are our variables for PCA and clustering in Lectures 3–5.

SQL and SQLite

A file-based engine for SQL

SQL — Structured Query Language

- Developed at IBM in the early 1970s
- ISO standard: every major database speaks SQL
- Declarative: say what you want, not how to get it
- Runs in PostgreSQL, MySQL, SQLite, BigQuery, DuckDB...
- Standard competency across research, government, and industry roles



*Useful: DB Browser for SQLite
(sqlitebrowser.org) — a free GUI for
exploring databases visually.*

Why SQLite for this course?

Everything lives in a single .sqlite file — trivial to share

- No server to install or configure
- The most-deployed database engine in the world (in every smartphone!)
- Handles datasets from MB to several GB comfortably
- Integrates with R via the DBI and RSQLite packages

2

Getting Connected in R

DBI · RSQLite · dbplyr

Installing Packages & Opening a Database

Three lines to get started

```
# Install packages (dplyr optional here) – only need to install this once
install.packages(c("DBI", "RSQLite", "dplyr", "dbplyr"))

# Initialize the libraries before using (everytime)
library(DBI)
library(RSQLite)

# Open a connection to the .sqlite file
# Data stays on disk: you're just sending queries to it
con <- dbConnect(SQLite(), "music.sqlite")

# What tables does the database contain?
dbListTables(con)
# [1] "albums" "artist_genres" "artists" "genres" "tracks"

# Pull an entire table into R as a data frame (output on console but can be
# captured with a 'table <- ' before!
dbReadTable(con, "artists")

# Always close the connection when done (routine always)
dbDisconnect(con)
```

con is a connection handle — data stays on disk. With large databases you never load everything into memory at once.

Two Ways to Query

Raw SQL and dbplyr

1 Raw SQL

```
result <- dbGetQuery(con, "  
  SELECT artist_id,name  
  FROM    artists  
  ORDER BY name  
  ")  
  
# result is a regular R data frame
```

Universal — this SQL works unchanged in PostgreSQL, BigQuery, DuckDB, and every other database engine.

2 dbplyr (tidyverse syntax)

```
library(dbplyr)  
  
artists_tbl <- tbl(con, "artists")  
  
artists_tbl %>%  
  select(artist_id, name) %>%  
  arrange(name) %>%  
  collect() # executes & returns a data frame
```

collect() is essential — it runs the query and brings the result into R memory. Without it, you just have a lazy reference.

*We can use both: raw SQL in the lab exercises, and dbplyr can be used when feeding results into R analysis workflows
Here: focus on SQL*

3

SQL Syntax

SELECT · WHERE · GROUP BY · JOINS · CASE WHEN

SELECT and FROM

The two words every query starts with – Note: the semicolon (;) not needed when working in RSQLite

```
-- Get everything from a table
SELECT * FROM artists;

-- Choose specific columns
SELECT title, popularity FROM tracks;

-- Rename a column in the output (alias)
SELECT title AS track_title, popularity FROM tracks;

-- Limit the number of rows returned
SELECT * FROM tracks LIMIT 10;

-- Select just the audio features of the 20 songs with highest
-- tempo
SELECT title, tempo, danceability, energy, valence
FROM tracks
ORDER BY tempo DESC
LIMIT 20;
```

Declarative

Describe what you want (not how the database should find it)

Column order

SELECT chooses which columns appear and in what order

Case-insensitive

SQL keywords are conventionally in CAPS — but lowercase works too

WHERE — Filtering Rows

Equivalent to dplyr's filter()

```
-- Exact match
SELECT * FROM tracks WHERE title = 'What is Love';

-- Numeric comparison
SELECT title, energy FROM tracks WHERE energy > 0.8;

-- Multiple conditions
SELECT title, valence, danceability
FROM tracks
WHERE valence > 0.7 AND danceability > 0.7;

-- Pattern match (% is the wildcard character)
SELECT title FROM tracks WHERE title LIKE '%love%';

-- Value in a list
SELECT title, valence, danceability
FROM tracks
WHERE title IN ('Invisible Love', 'What is Love', 'Blind Love');

-- Range
SELECT title, tempo FROM tracks WHERE tempo BETWEEN 120 AND 140;

-- Missing values
SELECT title FROM tracks WHERE acousticness IS NULL;
```

Operators (can also use brackets)

=	equal to
<> or !=	not equal
> < >= <=	numeric compare
AND / OR / NOT	combine conditions
LIKE '%x%'	pattern match
IN (...)	value in list
BETWEEN a AND b	inclusive range
IS NULL	test for missing

ORDER BY and LIMIT — Sorting Results

And the fixed order SQL clauses must follow

```
-- 20 most popular tracks
SELECT title, popularity
FROM   tracks
ORDER BY popularity DESC
LIMIT  20;

-- 10 tracks with the lowest energy (ascending energy)
SELECT title, energy
FROM   tracks
ORDER BY energy ASC
LIMIT  10;

-- Sort by two columns
SELECT album_id, title, tempo
FROM   tracks
ORDER BY album_id ASC, tempo DESC
LIMIT 20;
```

ASC = ascending (A→Z, low→high) · DESC = descending (Z→A, high→low)

Clause order

SQL clauses must appear in this sequence. You don't need all of them, but those you use must keep this order:

```
SELECT    -- which columns
FROM      -- which table
WHERE     -- filter rows
GROUP BY  -- aggregate
HAVING    -- filter groups
ORDER BY  -- sort
LIMIT     -- cap rows
```


GROUP BY — Aggregation

Compute summaries across groups of rows

```
-- How many explicit (1) and non-explicit (0) are there?
SELECT explicit, COUNT(*) AS n_tracks FROM tracks GROUP BY
explicit;
```

```
-- Average audio features per album
SELECT album_id,
       AVG(energy)      AS mean_energy,
       AVG(danceability) AS mean_dance,
       AVG(valence)     AS mean_valence
FROM   tracks
GROUP BY album_id;
```

```
-- Number of tracks per album, only 20 albums with the most
SELECT album_id, COUNT(*) AS n_tracks
FROM tracks
GROUP BY album_id
HAVING COUNT(*) >=5
ORDER BY COUNT(*) DESC
LIMIT 20;
```

Aggregate functions

COUNT(*)	number of rows
SUM(col)	total
AVG(col)	mean / average
MIN(col)	minimum value
MAX(col)	maximum value

WHERE vs HAVING

WHERE filters individual rows before grouping.

HAVING filters groups after aggregation.

You cannot write: WHERE COUNT(*) > 5

JOINS — Combining Rows from Multiple Tables

The most powerful concept in SQL

A JOIN matches rows from two tables based on a shared column, which is usually a foreign key to a primary key.

albums

album_id	artist_id	title
1	42	A Night at the Opera
2	42	News of the World
3	17	Like a Prayer

artists

artist_id	name
17	Madonna
42	Queen
88	Bowie (no albums)

JOIN
ON artist_id

result (INNER JOIN)

title	artist_id	name
A Night at the Opera	42	Queen
News of the World	42	Queen
Like a Prayer	17	Madonna

Bowie (artist_id 88) has no albums — not in result. That is INNER JOIN behaviour.

INNER JOIN — only rows that match in both tables. **LEFT JOIN** — all rows from the left table; unmatched rows from the right table receive NULL values.

JOINS in Practice — The SQL Syntax

Table aliases keep multi-table queries readable

```
-- Which country is each album's artist from?
SELECT  al.title    AS album,
        ar.name     AS artist
FROM    albums AS al
INNER JOIN artists AS ar
        ON al.artist_id = ar.artist_id;

-- LEFT JOIN: keep all albums even if artist data is missing
SELECT al.title, ar.name
FROM    albums al
LEFT JOIN artists ar ON al.artist_id = ar.artist_id;

-- Three-table join: track → album → artist
SELECT t.title    AS track,
        al.title   AS album,
        ar.name    AS artist
FROM    tracks t
INNER JOIN albums  al ON t.album_id   = al.album_id
INNER JOIN artists ar ON al.artist_id = ar.artist_id
LIMIT 20;
```

Aliases (al, ar, t) are set with AS or just a space after the table name. They make long queries far more readable

A Five-Table Query — Audio Features by Genre

This query is the bridge from SQL to our EDA work

```
-- Average audio features for every genre
-- Joins all five tables in the music database
```

```
SELECT  g.name                AS genre,
        AVG(t.energy)         AS mean_energy,
        AVG(t.danceability)   AS mean_dance,
        AVG(t.valence)        AS mean_valence,
        AVG(t.tempo)          AS mean_tempo,
        COUNT(t.track_id)     AS n_tracks
FROM tracks t
INNER JOIN albums  al  ON t.album_id  = al.album_id
INNER JOIN artists ar  ON al.artist_id = ar.artist_id
INNER JOIN artist_genres ag ON ar.artist_id = ag.artist_id
INNER JOIN genres  g   ON ag.genre_id  = g.genre_id
GROUP BY g.name
ORDER BY mean_energy DESC;
```

Good way to understand this:

```
SELECT *
FROM tracks t
INNER JOIN albums al ON t.album_id = al.album_id
INNER JOIN artists ar ON al.artist_id = ar.artist_id
INNER JOIN artist_genres ag ON ar.artist_id = ag.artist_id
INNER JOIN genres g ON ag.genre_id = g.genre_id
```

Generates a huge csv! Subsequent GROUP BY and the averages produces what we want!

This returns a genre × feature matrix: which is exactly the input format for PCA and clustering.

CASE WHEN — Conditional Logic in Queries

Create categorical labels from continuous variables

```
-- Categorise tracks by energy level
SELECT title, energy,
       CASE
         WHEN energy > 0.7 THEN 'high'
         WHEN energy > 0.4 THEN 'medium'
         ELSE 'low'
       END AS energy_cat
FROM tracks;

-- Mood quadrant: energy x valence
SELECT title, energy, valence,
       CASE
         WHEN energy > 0.6 AND valence > 0.6 THEN 'Happy & Energetic'
         WHEN energy > 0.6 AND valence <= 0.6 THEN 'Tense'
         WHEN energy <= 0.6 AND valence > 0.6 THEN 'Peaceful'
         ELSE 'Melancholic'
       END AS mood_quadrant
FROM tracks;
```

Useful for:

- Creating ordinal labels from continuous variables
- Recoding numeric codes into readable categories
- Handling or flagging missing data
- Preparing grouping variables for downstream EDA

4

Lab 1

Hands-on querying of the music database

Lab 1 — Exploring the Music Database

Work individually or in pairs

Preparation

- Download R scripts from courseroom on Lisam
- Download music.sqlite
- Prepare project and folder structure (place sqlite in /data/ folder)

Tutorial

sql_tutorial.R

- Work through the R script
- Read comments, follow reasoning
- Execute SQL queries
- Check results
- Indulge and learn! Fun stuff!

Exercises

sql_exercises.R

- 9 exercises where you create your own SELECT statements
- Use the Tutorial and slides material
- Ask questions
- PERFECTLY FINE if some things are tricky

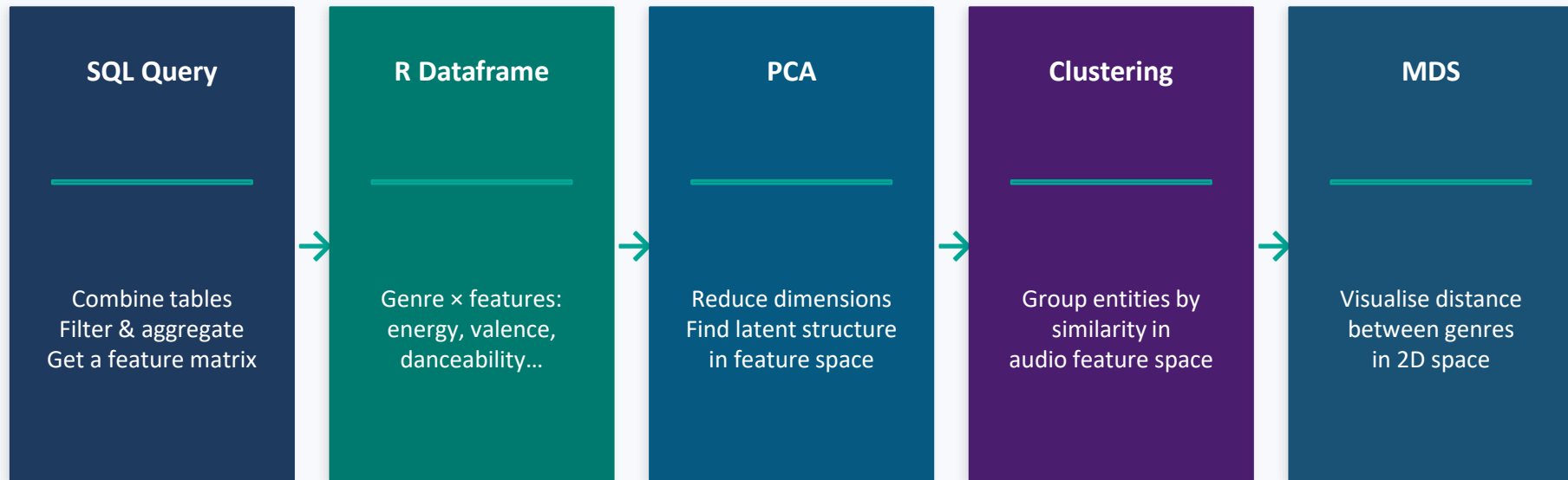
Solutions

sql_exercises_correct.R

- Check through the solutions
- See if you can untangle how the SQL statements work

The Big Picture — From SQL to Exploratory Data Analysis

What today built, and where it leads



The data you queried today in SQL is something we will return to for PCA and clustering

Friday: Lecture 2 — thinking about data structures, designing your own database schema and create database.